

```

from collections import deque

def vacuum_cleaner():
    # State: (vacuum_position, room_A, room_B)
    # 0 = Dirty, 1 = Clean

    initial_state = ("A", 0, 0)
    goal_state = (None, 1, 1)

    queue = deque()
    queue.append((initial_state, []))

    visited = set()

    while queue:
        state, path = queue.popleft()

        if state in visited:
            continue

        visited.add(state)
        path = path + [state]

        position, room_A, room_B = state

        # Goal test
        if room_A == 1 and room_B == 1:
            return path

        next_states = []

        # Suck action
        if position == "A" and room_A == 0:
            next_states.append(("A", 1, room_B))

        if position == "B" and room_B == 0:
            next_states.append(("B", room_A, 1))

        # Move Right
        if position == "A":
            next_states.append(("B", room_A, room_B))

        # Move Left
        if position == "B":
            next_states.append(("A", room_A, room_B))

        for next_state in next_states:

```

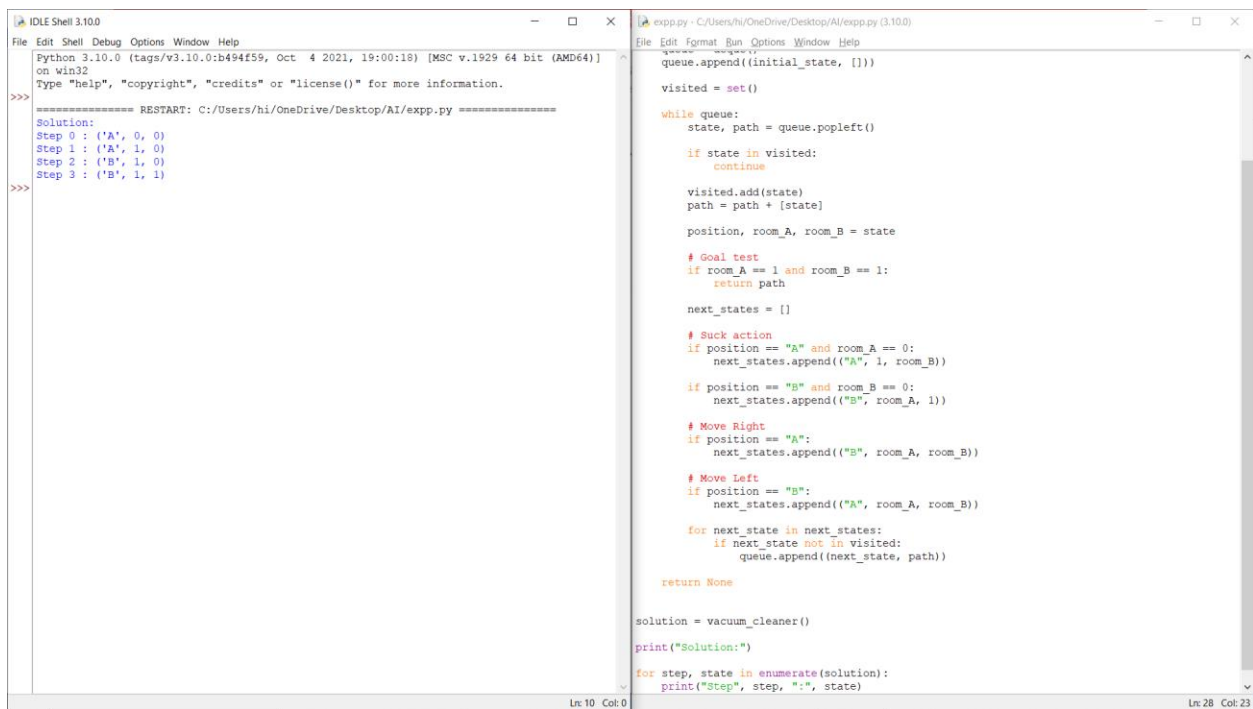
```
    if next_state not in visited:
        queue.append((next_state, path))
```

```
    return None
```

```
    solution = vacuum_cleaner()
```

```
    print("Solution:")
```

```
    for step, state in enumerate(solution):
        print("Step", step, ":", state)
```



The image shows two side-by-side windows from a Python IDE. The left window, titled 'IDLE Shell 3.10.0', displays the output of a program. It shows a restart message, a solution path, and four steps of the vacuum cleaner's movement. The right window, titled 'expp.py - C:/Users/hi/OneDrive/Desktop/AI/expp.py (3.10.0)', shows the source code for the program. The code defines a function to find a solution path for a vacuum cleaner in a two-room environment, using a breadth-first search algorithm. It includes actions like sucking, moving right, and moving left, and checks for goal states and visited states.

```
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/hi/OneDrive/Desktop/AI/expp.py =====
Solution:
Step 0 : ('A', 0, 0)
Step 1 : ('A', 1, 0)
Step 2 : ('B', 1, 0)
Step 3 : ('B', 1, 1)
>>>
```

```
expp.py - C:/Users/hi/OneDrive/Desktop/AI/expp.py (3.10.0)
File Edit Format Run Options Window Help
queue.append(initial_state, [])

visited = set()

while queue:
    state, path = queue.popleft()

    if state in visited:
        continue

    visited.add(state)
    path = path + [state]

    position, room_A, room_B = state

    # Goal test
    if room_A == 1 and room_B == 1:
        return path

    next_states = []

    # Suck action
    if position == "A" and room_A == 0:
        next_states.append(("A", 1, room_B))

    if position == "B" and room_B == 0:
        next_states.append(("B", room_A, 1))

    # Move Right
    if position == "A":
        next_states.append(("B", room_A, room_B))

    # Move Left
    if position == "B":
        next_states.append(("A", room_A, room_B))

    for next_state in next_states:
        if next_state not in visited:
            queue.append(next_state, path)

    return None

solution = vacuum_cleaner()

print("Solution:")

for step, state in enumerate(solution):
    print("Step", step, ":", state)
```